

Tiny machine learning for hyperspectral skin cancer image classification

Bernardo Petracchi 

University of Pavia, Italy

ARTICLE INFO

Keywords:

Machine learning
Hyperspectral images
GPU
FPGA
Tiny machine learning
Artificial neural networks

ABSTRACT

As the complexity of Machine Learning (ML) models grows, the need for powerful and high-memory devices is becoming more and more pressing. Furthermore, some scenarios like medical environments frequently deal with high-dimensional data such as clinical images that are challenging to analyze and elaborate using ML models without suitable hardware. This is the case, for instance, of hyperspectral (HS) images offering a wealth of spectral information that can aid in early detection and characterization. Many studies present in the literature address the use of Graphical Processing Units (GPUs) and Field Programmable Gate Arrays (FPGAs) for elaboration and classification of HS images in both the medical and non-medical field obtaining relevant results at the expense of a high-power consumption and a long time for designing. This paper aims at introducing an emerging subfield of ML called Tiny Machine Learning (Tiny ML) including hardware and software able to perform data analysis on extremely low-power devices such as microcontrollers, typically in the mW range or below. Furthermore, the research described in this paper wants to highlight how complex models like Artificial Neural Networks (ANNs) can be executed on portable devices such as the Arduino Nano 33 BLE (Bluetooth Low Energy) Sense module to perform low-cost HS pre-screening of skin lesions. Results show the effectiveness of this new branch of Artificial Intelligence (AI) in classifying HS skin cancer images.

1. Introduction

Nowadays, we continuously come across devices or systems that make use of Artificial Intelligence (AI) to support people in carrying out several tasks or making decisions. Some of these technologies are of everyday usage and include voice assistants, chatbots, smart televisions, advanced driver assistance systems, real-time translators and many others. Concerning more specific environments like hospitals, intelligent systems helping doctors during the diagnosis phase or surgical procedures are increasingly used. Whatever the application scope of these systems, they are all based on Machine Learning (ML) models. Nevertheless, complex ML models, especially Artificial Neural Networks (ANNs), require significant resources both in terms of memory and power. For this reason, executing such models on local and resource-constrained platforms is really challenging. A solution to this limitation is the use of cloud servers so that the user only has to deal with software development without having to manage hardware resources. Depending on the complexity of his application, he can rent the most suitable hardware platform available on the cloud and pay just for the time needed according to the *pay as you go* policy. In particular, a subfield of ML called Cloud ML aims at improving latency and throughput on cloud servers by targeting high-throughput accelerators like

Graphical Processing Units (GPUs) [1]. However, the main issue of cloud computing is that the user has to completely rely on the internet connection that, in case of slowdowns, prevents answers from being immediately available. Moreover, the transfer of data to remote servers could jeopardize customers' safety and privacy. All the aforementioned limitations make the employment of remote servers impracticable in critical environments like hospitals, where patients can need a real-time diagnosis and their sensitive data should be protected. In contraposition to Cloud ML, Edge ML focuses on portable devices like mobile phones and aims at improving energy efficiency, latency and preserving privacy on edge devices [1]. As expected by [2], the environment where we live is increasingly surrounded by connected resource-constrained devices able to implement accurate predictions and make decisions by means of ML models. The faculty of incorporating ML models on these edge devices prevents the data from being transferred to a central server for inference decreasing latency and limiting the network congestion. Recently, the scope of Edge ML has been extended to ultra-low-power devices like microcontrollers and Internet of Things (IoT) devices; this new paradigm of ML is called Tiny ML [1]. The term Tiny ML widely refers to a lightweight ML for embedded devices, where the ultra-low power consumption (typically below 1 mW) plays an essential role [3, 4]. The big advantage of this promising technology is that it employs

E-mail address: bernardo.petracchi01@universitadipavia.it.

<https://doi.org/10.1016/j.knosys.2026.116548>

Received 13 June 2025; Received in revised form 7 June 2026; Accepted 24 June 2026

Available online 25 June 2026

0950-7051/© 2026 The Author(s). Published by Elsevier B.V. This is an open access article under the CC BY license (<http://creativecommons.org/licenses/by/4.0/>).

widespread devices like microcontrollers that are cheaper and easier to program than other platforms such as GPUs and Field Programmable Gate Arrays (FPGAs). Nevertheless, the main challenge of Tiny ML is the limited computational resources of microcontrollers that hamper the development of large ML models. Fortunately, light software frameworks, tools, and libraries supporting Tiny ML models are available, especially for use with microcontrollers [3]. The typical workflow of Tiny ML includes the ML model development and training phase on an efficient computer; then the ML model undergoes optimization operations to be transferred to a microcontroller for inference [5]. Several existing works in the literature have already explored the field of Edge ML applied to healthcare applications [6,7]. In [6], an overview of the most relevant studies addressing the employment of edge computing platforms like Raspberry Pi and smartphones to implement ML is given. In particular, the findings reported in this work strongly focus on ANNs, especially Convolutional Neural Networks (CNNs) and Long Short-Term Memory Networks (LSTMs). The work presented in [7] describes the classification of electrocardiography (ECG) biosignals by means of Deep Learning (DL) models implemented on an NVIDIA Jetson device.

The purpose of this manuscript is to apply the subfield of Edge ML, namely Tiny ML, to a set of hyperspectral (HS) images related to skin lesions. For this reason, the Arduino Nano 33 BLE (Bluetooth Low Energy) Sense board was used enabling a new class of portable pre-screening tools featuring extremely low-power consumption (approximately a few tens of milliwatts). While a PC-based solution would significantly increase system complexity, cost, and power requirements, the proposed solution provides simplified system architecture, and reduced maintenance requirements. Furthermore, relying on a general-purpose operating system may introduce additional complexity and reliability issues. Last but not least, this work wants to provide a low-cost and portable system for skin lesion pre-screening easily accessible even in regions where access to hospitals and advanced medical facilities is limited (e.g. rural clinics). In this context, the main goal is to support healthcare personnel in a preliminary analysis of potentially suspicious lesions and suggest further clinical examination, whereas the final diagnosis remains under clinician responsibility. This work shows how the combination of Tiny ML and HS imaging (HSI) applied to portable devices represents a feasible low-cost approach for HS skin cancer pre-screening under resource-constrained scenarios.

HS images contain the spectral information of every pixel over a specific wavelength range [8] and are acquired with a spectrometer, called HS camera, that is capable of collecting scenes concerning hundreds of spectral bands both within the visible and non-visible range. HSI measures the fraction of the incident electromagnetic radiation reflected by the surface (reflectance) allowing precise material identification [9]. Each material features a unique variation of reflectance over the wavelength that is called spectral signature [10]. In short, acquiring images with an HS camera provides much more information compared to traditional RGB cameras and can lead to better performances [11]. In medicine, HSI is adopted as diagnostic support [12–14] and surgery guidance [15–17]. Concerning skin cancer diagnosis, HSI represents a valid alternative to biopsy, that is an onerous and time-consuming process [18–20]. Since HSI is a non-invasive and non-ionizing technique, it became the purpose of study in medicine for cancer detection [21]. The literature shows that ML and DL models represent ideal solutions in classifying medical HS skin cancer images [8,11,22–31] and highlights how the former are suitable when the dimensions of the training dataset is limited, while the latter perform better in case of a higher number of training samples. Specifically, the works reported in [8,22,29–31] classified the same HS dataset of this paper, but focused on powerful platforms like Intel processors, NVIDIA GPUs or FPGAs without considering tiny or edge devices. The employment of edge devices in the HSI field can be found in [21,32,33]. In [21], the authors proposed an automatic HSI system composed of a HS camera and a movement system consisting of a linear ball screw guide, a stepping motor and a driver controlled by an Arduino Uno board. Concerning skin

cancer applications, this system allows to acquire HS data through the automatic movement of the camera while maintaining the sample stationary. Similarly, the work described in [32] concerns a low-cost HS scanner for natural imaging in both terrestrial and underwater environments. The scanner is based on a commercial spectrometer and two movable UV reflecting mirrors directing the light from the scanned scene onto the spectrometer. An Arduino Uno microcontroller is employed to iteratively move the two mirrors and to gradually form an image. In [33] a portable dicot leaves high-resolution HSI imager named LeafSpec using an Arduino nano as the microcontroller and a Raspberry Pi 4 as the microprocessor was developed. Nevertheless, in all these works an Arduino device is used as a support in a HS acquisition system but not for data elaboration or performing inference. The aim of this paper is to use an edge device, namely the tiny Arduino Nano 33 BLE Sense module, to implement a DL model and perform inference on a HS skin cancer dataset. This paper is structured in the following way. Section 2 presents the current DL and AI developments employed in the HSI field. After a description of the acquisition system to capture the HS images, Section 3 describes the skin cancer database. Then, the architecture of the DL model and the tiny device adopted to classify the HS dataset are illustrated. Moreover, the techniques used to port the DL model into the tiny device are presented. Section 4 shows the results, whereas in Section 5 the discussions are presented. Finally, Section 6 draws the conclusions and proposes future developments. The main contributions of this manuscript are the following:

- (i) The demonstration of the feasibility of portable HS pre-screening devices entirely executed on an ultra-low-power Arduino microcontroller;
- (ii) The design of a sequential processing pipeline that enables the handling of HS data under strict memory constraints, avoiding the need to store the entire image in external memories or buffers;
- (iii) The implementation of a pixel-wise inference strategy tailored for embedded deployment, allowing the processing of thousands of spectral vectors per image within the limited RAM and Flash resources available on the device;
- (iv) The description of the conversion and quantization techniques to port complex models into tiny devices.

2. Limitations in HS skin cancer classification

Despite the great progress made in HSI for skin cancer analysis, several limitations still remain, hampering the development of robust models. First, HS skin cancer datasets are typically limited in size due to the complexity of data acquisition, and their public availability is constrained by the need to preserve patients' privacy. The shortage of publicly available large medical datasets typically leads to overfitting, that is not desired in healthcare [34]. Furthermore, the high cost of the image annotation stage exacerbates this limitation since it requires experts such as dermatologists or pathologists. Moreover, spectral signatures are highly sensitive to inter-patient variability, acquisition sensors, and illumination conditions, which decreases the robustness of trained models across different clinical scenarios. The lack of standardization across HSI systems is underlined in [35] and hampers the flexible reuse of pretrained models. Another critical issue is spectral mixing, particularly at lesion boundaries where a single pixel may contain both lesion and healthy skin information, leading to classification errors. Finally, HS images provide ample information about the observed scene since they can feature hundreds of bands leading to a high probability of redundancy due to the high band correlation [34]. This last aspect together with the shortage of labelled data is not limited to medicine, but also affects many fields related to HSI.

Recent advances in HSI applied to different practical domains are enabled by DL and AI-based methods. A comprehensive overview of such progress is reported in [35]. Specifically, [35] introduces an overview of the most recent research in many tasks including anomaly

detection and remote sensing applications. Among the proposed solutions, foundation models emerged as the ideal candidates also in case of scarcity of labelled samples. Indeed, once pretrained, these models require small labelled datasets for fine-tuning. Moreover, foundation models address the challenge of discriminative feature extraction from redundant data. Some works in the literature already employed these models effectively. For instance, [36] introduces an innovative Visual Foundation Model (VFM) for Earth observation called SeaMo that effectively integrates multimodal and multi-seasonal remote sensing information through transformer-based architectures and masked image modeling strategies.

Similarly, SpectralGPT [37] is able to handle spectral remote sensing images leveraging a novel 3D generative pretrained transformer (GPT) based on 3D cube masking to capture compact spatially and spectrally visual representations in an effective way. Furthermore, an encoder extracts latent features from partially observed spectral cubes, whereas a decoder reconstructs the missing information.

In HS anomaly detection, recent strategies such as the method proposed in [38] combine model-driven and data-driven paradigms through the learning of disentangled priors. In [38], low-rank representation (LRR) models are integrated with Deep Neural Networks (DNNs) to better characterize background structures while separating anomalous components. LRR assumes that the background pixels are similar and their spectra are correlated. As a consequence, a lot of information is redundant and can be compressed into a few main components.

Although foundation models and hybrid data-driven/model-driven approaches represent state-of-the-art methods in HSI, they still suffer from high computational requirements, limited interpretability, and reduced generalization [39] across different acquisition systems and clinical settings. Also, these models are typically executed on high-end CPUs and GPUs [36,37], making their deployment challenging in portable and resource-constrained platforms such as the one adopted in this study. Conversely, the proposed method targets an ultra-low power platform constrained by limited computational resources and memory. The main contribution of this work is not the development of a novel large-scale DL architecture, but rather the demonstration of HS skin cancer portable pre-screening feasibility on ultra resource-constrained hardware. To the best of our knowledge, very few works in the literature address HS skin cancer inference entirely executed on a tiny device. Finally, the proposed method represents a trade-off between classification accuracy, memory usage and computational complexity.

3. Materials and methods

3.1. The hyperspectral acquisition system and the skin cancer dataset

The HS skin cancer dataset considered in this work was not directly acquired by the proposed system, but it was collected by the authors of [23]. Therefore, the sensing stage including the HS camera and the acquisition process, is not part of the proposed pipeline. The acquisition system used in [23] to collect the HS images is based on a snapshot camera (Cubert UHD, Cubert GmbH, Ulm, Germany) covering the 450–950 nm wavelength range, distributed over 125 spectral channels. The spectral resolution is 8 nm, whereas the spatial resolution is 50 50 pixels; the pixel size is 240 240 μm^2 . A Dolan-Jenner halogen source light (Dolan-Jenner, Boxborough, MA, USA) and a 150 W quartz-tungsten bulb provide the illumination. The choice of a snapshot sensor lies in its speed since it acquires the whole images through a single exposure [28,30]. The acquisition system also includes a fibre optic ring light guiding the HS camera to illuminate the skin surface with cold light, avoiding the high temperature of the halogen lamp on the subject's skin. Moreover, the system was equipped with a 3D-printed contact structure embedding a dermoscopic lens with a human skin refraction index. The process of capturing an HS image is controlled by an acquisition software and takes 250 ms [23]. The HS skin cancer dataset, that is

also used in [8,22,23,29–31], comprises 76 images of skin lesions related to 61 subjects and its acquisition campaign occurred from March 2018 to June 2019 in the Hospital Universitario de Gran Canaria Doctor Negrín and the Complejo Hospitalario Universitario Insular-Materno Infantil (Canary Islands, Spain). The labelling of the images was carried out by experts such as dermatologists and pathologists through biopsy-proven histological assessment of the suspected malignant lesions. The resulting classes of the lesions are the following: Benign Epithelial (BE), Benign Melanocytic (BM), Malignant Epithelial (ME), and Malignant Melanocytic (MM). Table 1 summarizes the percentage distributions of the skin lesions including the four possible categories. We can observe that the BM class is the most numerous (45%), whereas the BE class presents, instead, the smallest number of samples (7%). Nonetheless, if we consider the overall number of malignant and benign lesions, the dataset results reasonably balanced: 48% of the lesions are malignant, whereas 52% of the lesions are benign.

Fig. 1 depicts the ground truth of all the 76 HS images associating a color per class. The BM and the BE classes are colored by green and blue respectively; ME and MM are represented with yellow and red colors. The remaining portion of the figure is in black to represent the healthy tissue (Skin). The skin cancer dataset, originally composed of 125 spectral channels, have undergone to some pre-processing steps to mitigate the variability among different patients and the variations in illumination condition. These steps include calibration, removal of the extreme bands, noise reduction, and pixels normalization. The calibration was applied following the formula of Eq. (1):

$$PI = \frac{RI - DI}{WI - DI} \quad (1)$$

RI represents the original raw HS image, WI is the white reference image, whereas DI is the dark reference image. Finally, PI is the post-calibration image. The acquisition of WI involved the use of a reference tile capable of reflecting 99% of the incident light. Instead, DI was acquired turning off the room light and closing the camera shutter.

Thereafter, due to the poor response of the HS camera in the extreme bands, the first four and the last five spectral channels have been removed. This latter step has been followed by the application of a moving average filter to reduce the noise according to Eq. (2):

$$y(i) = \frac{1}{N} (y(i+N) + y(i+N-1) + \dots + y(i-N)) \quad (2)$$

$y(i)$ represents the new value assigned to pixel i , whereas N is the number of neighbor pixels considered that in this work was 5. Finally, the span of the filter is $2N + 1$.

As conclusion of the pre-processing phase, the min-max procedure was adopted to perform pixels normalization according to Eq. (3):

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}} \quad (3)$$

Where x' is the normalized value of pixel x , whereas $x_{\min}$ and $x_{\max}$ are the minimum and the maximum values among all the spectral channels for pixel x .

The pre-processing results are reported in Fig. 2 that shows the superposed average spectral signatures of each class obtained after the pre-processing phase.

Table 1
Percentage distribution of each skin lesion.

Skin lesion	Percentage distribution [%]
BE	7
BM	45
ME	32
MM	16

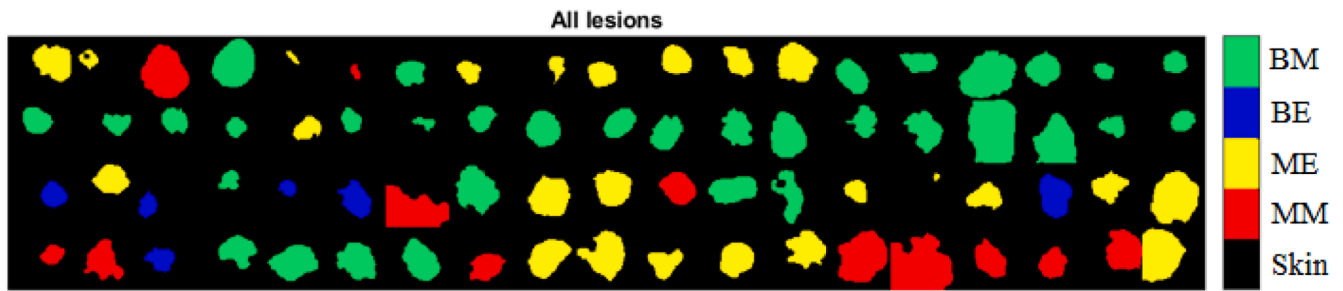


Fig. 1. Ground truth of the 76 HS images.

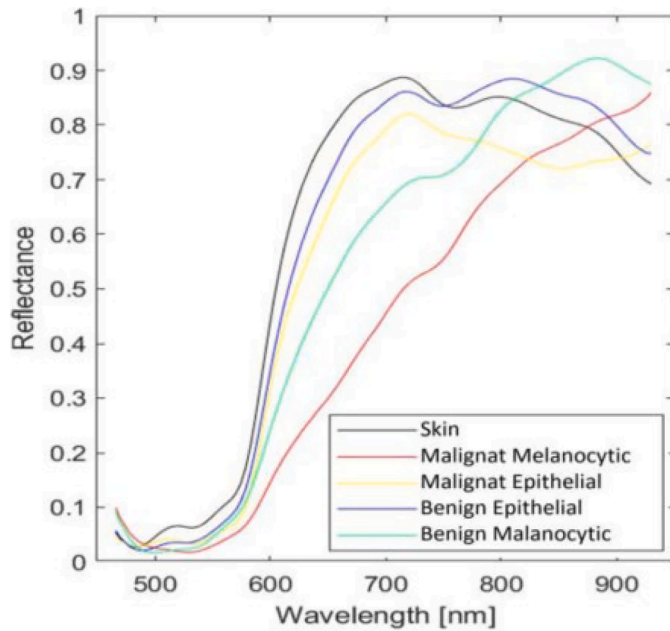


Fig. 2. Average spectral signatures of the healthy tissue and of the four classes related to the skin cancer database [22].

3.2. Convolutional neural network architecture

The model adopted to classify the HS skin cancer images was a CNN, an advanced version of ANN mainly designed to extract features from grid-like matrix datasets such as images or videos. Thanks to their effectiveness in processing visual data, CNNs are widely used in computer vision applications like image classification, semantic segmentation, object detection, and image super-resolution reconstruction [40]. Concerning the HSI field, the works proposed in [41–44] confirm the CNNs suitability due to the high classification accuracy obtained. The CNN employed in this paper was developed using Google Colab, a hosted Jupyter Notebook service that requires no setup to use and provides free access to computing resources, including CPUs, GPUs and Tensor Processing Units (TPUs). The code was written in Python language using the open-source TensorFlow library that supports ML models development. The architecture adopted in this work is a 1D CNN specifically designed to process one-dimensional data like the HS pixels of this work. Indeed, each HS pixel to classify is represented by an array of 116 float values. It is worth noting that spatial-context or segmentation-oriented methods were not adopted in this work. As reported in [45], classifiers exploiting spatial information typically achieve better performance than pixel-wise approaches. Nevertheless, such methods typically rely on spatial patches, which require significantly more RAM and processing power making the inference process slow and more challenging due to the limited memory and computational resources of the Arduino Nano 33

BLE Sense. For this reason, a pixel-wise strategy was deliberately adopted as a trade-off between accuracy and computational efficiency. A similar consideration applies to [23], where the authors first perform image segmentation using K-means clustering and spectral angle mapper algorithm (SAM), then, they apply ML algorithms (like Support Vector Machine (SVM) that works well with few samples) for supervised classification. Once again, this pipeline increases the computational complexity hampering the development on resource-constrained devices. Although a hybrid approach where segmentation is performed offline and only classification is executed on the microcontroller could reduce computational requirements. However, such a solution would not represent a fully embedded system. The objective of this work is to explore the feasibility of performing HS inference entirely on a resource-constrained device, without relying on external processing units. Moreover, it would no longer be known whether a classification error with Arduino is due to a previous segmentation error or is intrinsic to the board. Instead, since pixel-level ground truth annotations are already available in the dataset, the pixel-wise strategy was considered sufficient in this study.

The proposed neural network classifies every pixel into one of the five possible classes (including healthy tissue). Then, the whole image is classified as malignant or benign by adopting a threshold-based approach similar to those used in [23]. Specifically, if the number of pixels classified as malignant exceeds a certain threshold (Th), the image is classified as malignant. According to [23], the value of Th chosen in this paper was 40%. This approach was deliberately selected due to its extremely low computational cost and its suitability for resource-constrained platforms, where more complex aggregation strategies (e.g., probabilistic modeling or segmentation-based methods) would significantly increase memory usage and processing time.

The CNN is composed of an input layer of 116 nodes covering all the 116 spectral bands of every pixel, followed by four convolutional layers with 32, 64, 32 and 32 filters respectively featuring a kernel size of 3 and ReLU activation function. To reduce the spatial dimensions, the first and the second convolutional layers are followed by max-pooling layers having a size of 2. After flattening the output, the latter was passed through two fully connected layers of 64 and 32 nodes respectively, once again featuring the ReLU function. The output layer is a softmax layer consisting of 5 nodes to calculate the pixel probability of belonging to the five classes. Fig. 3 shows the CNN architecture employed in this work, where FC1 and FC2 denote the first and the second fully connected layer. Concerning the training phase of the network, the whole dataset was divided by image into training dataset, validation dataset and test dataset composed of 46, 15 and 15 images respectively using stratified sampling due to the unbalanced classes. Image-level splitting preserves independence between training and test data, preventing pixels of the same image from being assigned to both the training and test set. Therefore, this splitting strategy allows a realistic assessment of model's generalization capability. Nevertheless, the train/validation/test split was not patient-independent. Model selection was performed on the validation set using repeated stratified 5-fold cross-validation with 5 repeats. The final model was then retrained on the combined

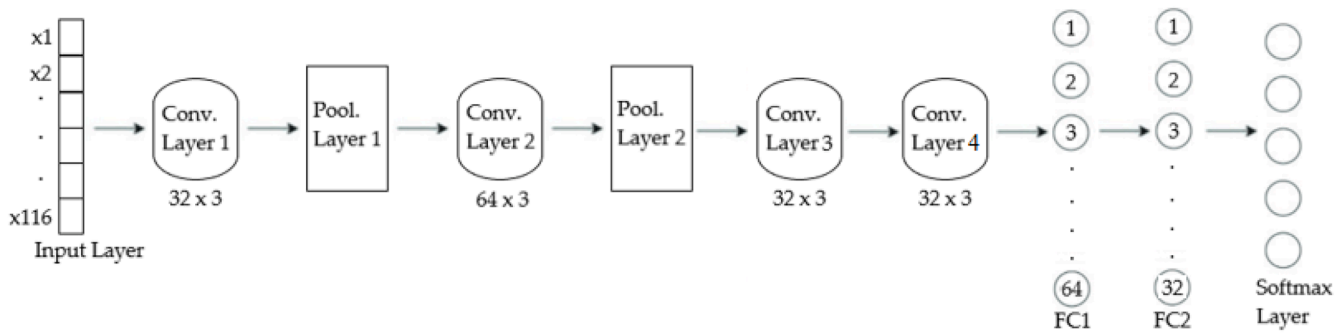


Fig. 3. Architecture of the 1D CNN.

training and validation sets and evaluated on the independent test set. Also, class weighting was applied during training to compensate for the imbalance, ensuring the model pays suitable attention to underrepresented classes. To adjust the weights during the training, the Adam optimizer with a learning rate of 0.0005 was adopted, while the categorical crossentropy loss function was used to evaluate how well the model was performing during the learning phase. The metric considered to assess the model's performance was accuracy. The model was trained for thirty epochs and with a batch size of 64 using an NVIDIA Tesla T4 GPU, a free-access device that Google Colab makes available with some usage limitations.

3.3. The Arduino nano 33 BLE sense module

The device used to classify the HS skin cancer dataset is the low-power Arduino Nano 33 BLE Sense miniature sized module. This module features a tiny form factor, being its dimensions of 18×45 mm. It is built upon the Nordic nRF52840 microcontroller, contains an Arm® Cortex®-M4F and a u-blox modem with BLE connectivity. Concerning memory, this module is equipped with a flash and a SRAM of 1 MB and 256 KB respectively. It provides a frequency clock of 64 MHz and supports different communication protocols such as Universal Serial Bus (USB), Serial Peripheral Interface (SPI), Inter Integrated Circuit (I2C) and Universal Asynchronous Receiver-Transmitter (UART). Thanks to its numerous environment sensors it can detect color, motion, proximity, humidity, temperature, audio and much more. Specifically, the board provides a 9-axis Inertial Measurement Unit (IMU) featuring a 3D accelerometer, gyroscope and magnetometer and allows orientation, motion or vibrations detection. The Arduino Nano 33 BLE Sense module can be programmed through the Arduino Integrated Development Environment (IDE) or using MicroPython that is an implementation of the Python programming language. Furthermore, the module offers the possibility to implement AI using Tiny ML and TensorFlow Lite. The latter is a toolkit supporting developers to run their models on

embedded, mobile, or IoT devices and enables them to implement on-device machine learning. It is precisely on this last characteristic that the work presented in this article is based. Last but not least, the Arduino Nano 33 BLE Sense supports low-power operation through hardware sleep modes of the nRF52840 microcontroller. According to the official Arduino documentation, standby current can be reduced to approximately 1 mA by disabling sensors and entering sleep states. This enables duty-cycled operation where the device remains inactive between inference events, reducing the average energy consumption. These features make this architecture an ideal match for portable devices and significantly more energy-efficient than PC-based solutions. Fig. 4 shows the Arduino Nano 33 BLE Sense module.

3.4. Model compression and quantization

After the model was built and trained using TensorFlow, it was converted into a lightweight and more efficient version called TensorFlow Lite model. A TensorFlow Lite model is represented in a special efficient portable format known as FlatBuffers, identified by the .tflite file extension and enabling efficient execution on resource-constrained devices. The TensorFlow Lite engine consists of two main components: a very light interpreter (with a size of <300 kB) able to run on different devices such as Android, iOS, and microcontrollers; and a converter that performs the conversion into the TensorFlow Lite model [46]. During the conversion process from a TensorFlow model to a TensorFlow Lite model, the size of the file is reduced. Nevertheless, the TensorFlow Lite model obtained after the conversion was 928 kB of memory, still too big to suit the limited resources of the Arduino Nano 33 BLE Sense module. For this reason, the model was subjected to a post-training quantization (PTQ) process to further reduce its size. PTQ is a conversion technique that can reduce model size while improving latency, with little degradation in model accuracy [47]. Through PTQ, the numeric precision of the model weights and activation values, originally in 32-bit floating point representation, was reduced to 8-bit integers representation after

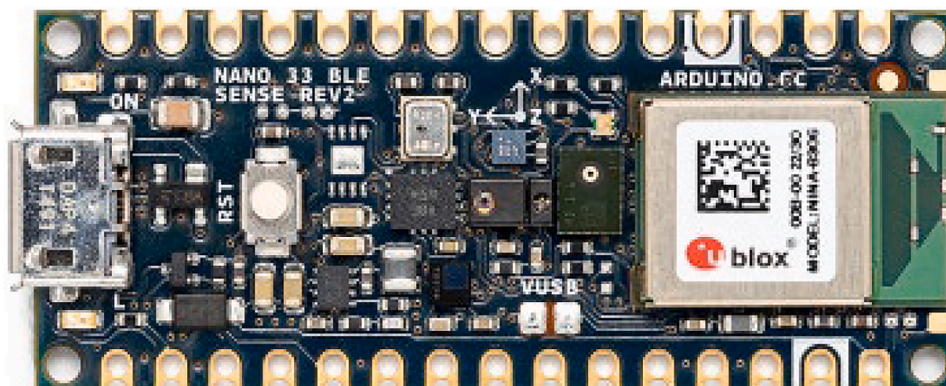


Fig. 4. The Arduino Nano 33 BLE Sense module.

the model training. However, whereas quantization of weights is straightforward, quantization of activation values is not and requires a calibration step to determine the scaling parameters [46]. Indeed, unlike weights, activation values are dynamic, that is, they change during inference and depend on inputs. Therefore, a calibration step to determine how to map continuous values into discrete values is strictly necessary. In this work, the calibration step was accomplished by creating a function that provides a representative subset of the input data. Then, TensorFlow Lite computed and stored the minimum and maximum values of the activations from the representative subset. Through the minimum and maximum values, TensorFlow Lite mapped any floating-point numbers in the $[-128, +127]$ interval. The activations are then quantized on the fly at inference time. After the quantization, the TensorFlow Lite model was reduced to 245 kB of memory that suits the resources of the Arduino module. In [Code 1](#), the model conversion and quantization processes are reported. Line 1 creates a *TFLiteConverter* object allowing to convert the original TensorFlow model (*model*) into a TensorFlow Lite model, whereas line 2 enables the default optimizations, namely, the quantization of weights and activations. From line 4 to line 7 the *representative_dataset_gen* function provides a dataset of 300 input samples (*pixel_data*). Line 9 associates the *representative_dataset_gen* function with the converter. Finally, in line 10 the out-and-out conversion is achieved.

The last step was to convert the quantized model into an array of bytes (in C language) in hexadecimal format and to save it into a header file with .h extension to be included in the Arduino module for inference; this was achieved through the *xxd* utility.

3.5. Arduino inference

The inference of the HS images was performed by developing an Arduino program, called sketch, through the Arduino IDE. [Code 2](#) reports a snippet of the Arduino sketch including the main operations adopted to perform the HS image inference on the Arduino Nano 33 BLE Sense board. In [Code 2](#), two pointers (*tfInputTensor* and *tfOutputTensor*) to the tensors used by TensorFlow Lite to handle the input and output data during inference are defined and initialized. In other words, *tfInputTensor* stores the HS pixel to be passed to the model during inference, whereas *tfOutputTensor* contains the inference results, that is, the HS pixel five probability values of belonging to each class. The same procedure is applied to the pointers to the model (*tfModel*) and to the interpreter (*tfInterpreter*). *tfOpsResolver* records all the operations supported by TensorFlow Lite. Then, the quantized model obtained in the previous section is included in the sketch through the *GetModel(model)* function, where the *model* parameter stores the model in hexadecimal format taken from the header file as discussed in [Section 3.4](#). It follows the *MicroInterpreter* object creation to allocate all the memory necessary for all tensors through the *AllocateTensors()* function, and to execute the model. *tensorArena* is an array of bytes designated to the TensorFlow Lite model execution; its dimension is specified through the *tensorArenaSize* variable that in this work is equal to 12 kB. Thereafter, all the pixel values contained in the *values* array are copied into the input tensor through a for loop that iterates over the 116 spectral bands

(*vectorLength*) of the pixel. Finally, the inference is performed by means of the *Invoke()* function.

4. Result

The CNN model, trained for thirty epochs, achieved accuracy values of 87% and 88% on the validation and the training dataset respectively. The training time was approximately four minutes using a T4 GPU of Google Colab. Then, the CNN was evaluated on the test dataset obtaining an accuracy of 76%, meaning that the model is able to make good predictions also regarding unseen data. The proposed architecture was selected after a tuning phase, where both simpler and more complex models have been evaluated. Nevertheless, the former led to scarce accuracy results, whereas the latter strongly slowed down the inference process slightly improving accuracy, or incurred memory issues. Other metrics related to pixel-wise classification evaluated on the test dataset were: F1 score, precision, recall, specificity, dice coefficient and Intersection over Union (IoU). The achieved values are reported in [Table 2](#). Concerning the image classification phase, twelve out of fifteen HS images of the test set have been correctly classified as malignant or benign, according to the threshold-based aggregation strategy described in [Section 3.2](#). This led to an image-level accuracy of 80% (95% CI: ~55%–93%). [Table 3](#) reports other image-level evaluation metrics including F1 score, recall, specificity, and precision, together with the confidence interval. The relatively wide confidence interval of the obtained metrics highlights the uncertainty of the evaluation due to the limited size of the test dataset. Unlike the other metrics, F1-score does not directly follow a simple binomial formulation, therefore, its uncertainty is more difficult to evaluate without adopting resampling-based approaches. For this reason, [Table 3](#) reports it as point estimate representing a qualitative comparison of the model performance rather than strict statistical inference. As shown in [Table 3](#), the recall value of 75% indicates that most malignant lesions are correctly identified. The specificity value of 86%, instead, shows that benign lesions are reliably recognized with a low number of false positive. Precision of 86% further confirms that when the system predicts a lesion as malignant, it is correct in most cases. Finally, the F1 score of 80% reflects a good balance between recall and precision. Nevertheless, two of the three misclassified images are related to the MM lesion. This is due to the unbalanced dataset containing few images of this kind. Overall, these results suggest that the proposed approach provides reliable image-level classification, although further improvements are needed to reduce false negatives and increase recall by employing a larger dataset.

Thereafter, the model was converted and quantized using the TensorFlow Lite toolkit in such a way to make it run on a tiny device like the Arduino Nano 33 BLE Sense module. To evaluate the HS image embedded inference, this device was used to perform a pixel-wise classification of a group of ten HS images belonging to the test set and containing all the possible skin lesions. Each image features a dimension of $50 \times 50 \times 116$, the latter being the number of spectral channels. The Arduino board was programmed through the Arduino IDE using the Arduino programming language that is based on C. The HS images were transferred from a regular laptop to the Arduino board through a serial

Code 1

TensorFlow Lite model conversion and quantization.

```
1: converter = tf.lite.TFLiteConverter.from_keras_model(model)
2: converter.optimizations = [tf.lite.Optimize.DEFAULT]
3:
4: def representative_dataset_gen():
5:     for i in range(300):
6:         pixel_data = inputs_test[i].reshape(1, 116, 1).astype(np.float32)
7:         yield [pixel_data]
8:
9: converter.representative_dataset = representative_dataset_gen
10: tf_lite_model = converter.convert()
```

Code 2

Fragment of Arduino sketch.

```

1: TfLiteTensor * tflInputTensor = nullptr;
2: TfLiteTensor * tflOutputTensor = nullptr;
3: const tflite :: Model * tflModel = nullptr;
4: tflite :: MicroInterpreter * tflInterpreter = nullptr;
5: tflite :: AllOpsResolver tflOpsResolver;
6:
7: tflModel = tflite :: GetModel(model);
8: tflInterpreter = new tflite :: MicroInterpreter(tflModel, tflOpsResolver,
9: tensorArena, tensorArenaSize);
10: tflInterpreter->AllocateTensors();
11:
12: for (int i = 0; i < vectorLength; i++) {
13: tflInputTensor->data.f[i] = values[i];
14: }
15:
16: TfLiteStatus invokeStatus = tflInterpreter->Invoke();

```

Table 2

Pixel-wise evaluation metrics obtained on the test dataset.

Accuracy [%]	F1 score [%]	Precision [%]	Recall [%]	Specificity [%]	Dice coef. [%]	IoU [%]
76	79	85	76	91	79	69

Table 3

Image-level evaluation metrics obtained on the test dataset.

F1 score [%]	Precision[%]	Recall[%]	Specificity[%]
80	86 (95% CI: ~49%–97%)	75 (95% CI: ~41%–93%)	86 (95% CI: ~49%–97%)

connection using a common micro-USB cable and a baud rate of 9600. The whole transfer process was handled by a Python script using the *serial* library. The time taken to classify a HS image was 2.30 min, whereas the mean accuracy value (over the ten images) obtained together with the standard deviation was $87.0\% \pm 10.5\%$. To have a fair comparison, the mean accuracy was calculated even for the Colab version, achieving a value of $82.6\% \pm 12.3\%$. The slightly higher accuracy observed on Arduino compared to Colab can be attributed to several factors. First, the quantization process introduces numerical approximations that may slightly affect the decision boundaries. Second, differences between floating-point (Colab) and fixed-point (Arduino) computations may lead to small variations in predictions. The fact that the employment of the quantized model did not lead to an accuracy reduction compared to the via-software implementation represents a significant result given the limited resources of the Arduino device adopted in this research. However, it is important to note that the evaluation is performed on a limited number of images, resulting in relatively high variability (± 10 – 12%). Therefore, the observed difference is not statistically significant and should be interpreted as comparable performance rather than a true improvement of one platform over the other. Moreover, the image-level accuracy was calculated even with the Arduino board, specifically: eight out of ten HS images have been correctly classified as malignant or benign, thus achieving a value of 80% (95% CI: ~49%–94%). This confirms the effectiveness of the board in classifying medical images and the suitability of the proposed aggregation strategy. A last important aspect concerns power

Table 4

Estimated values of power and energy consumption, and memory usage.

P[mW]	E_{im} [J]	E_{pix} [mJ]	Flash[KB]	RAM[KB]
~20	~3	~1.2	491	12

consumption and memory usage in performing the inference. Table 4 reports the estimated values of the instantaneous power employed during the model execution (P), the energy per image (E_{im}), the energy per pixel (E_{pix}), the flash memory occupied by the quantized TensorFlow Lite model (converted in C array), and the RAM occupied during inference.

To provide some visualized results, Fig. 5 shows the Arduino and Colab predicted masks of three images taken from the test set, together with their corresponding ground truth. The colors blue and red represent benign and malignant lesions respectively, whereas color black represents the healthy tissue. In addition, pixel wise error maps are provided to highlight the discrepancies between predictions and ground truth (in this case color red represents all the misclassified pixels regardless of lesion type). We can observe that both Arduino and Colab predict most of the pixels correctly in case of Image ME and Image BM, whereas they fail in case of Image MM. Indeed, in this latter case the models misclassify most of the lesion pixels leading to many false positives. This behavior can be attributed to class imbalance of the considered dataset. It is also worth noting that the ground truth of this image contains a non-homogeneous lesion structure with healthy skin pixels inside the lesion region. In this specific case, the Arduino model appears to better preserve these internal healthy regions, whereas the Colab model tends to over-segment the lesion area. Finally, most misclassifications are concentrated at lesion boundaries. This is expected in HSI due to spectral mixing effects, where boundary pixels contain a mixture of lesion and healthy skin signatures, leading to ambiguous spectral responses.

5. Discussion

Some studies in literature have already developed ML and DL models to classify the same HS dataset considered in this work [8,22,29–31]. All these works adopted the same preprocessing steps described in Section 3.1. Nevertheless, details regarding the dataset splitting strategy (pixel-level vs image-level) are not explicitly provided. Therefore, a fully fair comparison cannot be guaranteed, and the benchmark should be interpreted with caution.

Specifically, the research carried out in [22] addresses three popular ML methods, namely, SVM, Random Forest (RF) and eXtreme Gradient Boosting (XGB) achieving a mean accuracy of 97% confirming the ML methods effectiveness when the dimension of the dataset is limited. Although the accuracy obtained in [22] is higher than the one achieved in this manuscript, it is necessary to highlight that to develop their works, the authors employed a PC equipped with an Intel i7–1065G7 CPU and did not suffer from limited hardware resources as it occurs in this work. In this context, the results achieved in this work represent a valid trade-off between model complexity and the limited resources of the hardware.

The authors of [30] developed a Vision Transformer (ViT), a kind of

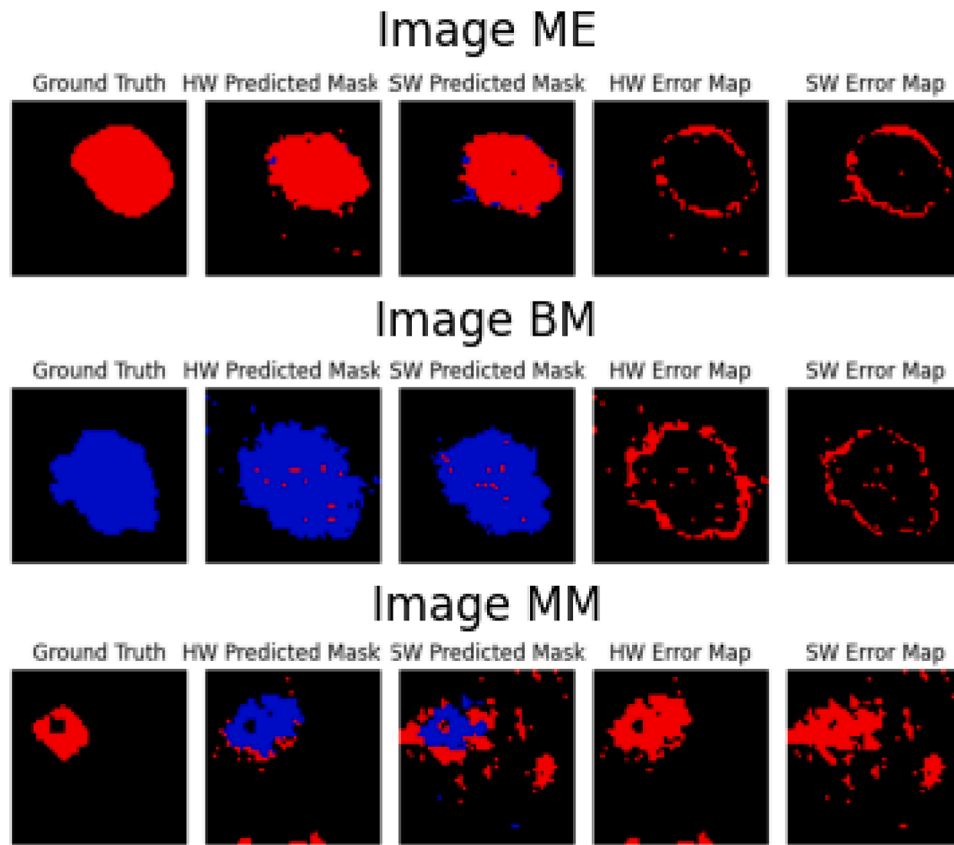


Fig. 5. Ground truth, predicted masks and error maps of three images taken from the test set.

ANN architecture that relies on the self-attention mechanism and designed for computer vision applications, obtaining a mean accuracy of 70%.

In [29], several DL architectures including ResNet-18, ResNet-50, ResNet101, 3D ResNet-50, U-Net, U-Net++ and two versions of the DeepLab V3+ have been developed. In particular, the U-Net++ resulted in the most performant architecture and achieved a mean accuracy of 63%.

In both [29] and [30], the experiments were conducted with a test system equipped with an Intel i9-9900C CPU and two 2944-core NVIDIA RTX 2080 GPUs. Additionally, the work in [29] adopted a low-power Jetson GPU. The authors of [29] stated a power consumption of 250 W with the RTX 2080 GPU and 10 W or 5 W (depending on the power budget set on the device) with the Jetson GPU. With equal inference time, the device used in this research consumes less energy, as reported in Table 4.

Table 5 summarizes the comparison between the metrics obtained in this work with those obtained in [22,29,30]. The objective of this comparison is not to claim superiority over high-performance platforms, but to highlight that good classification performance can be achieved even under severe hardware constraints. Given the differences in the evaluation protocols and hardware platforms, the comparison reported

in Table 5 should be interpreted as qualitative rather than strictly quantitative.

The research carried out in [8] concerns the parallelization of the same ML methods adopted in [22] to perform the HS skin cancer image inference on different NVIDIA GPU platforms including an RTX 2080, an RTX 4090 and a cluster composed of five nodes of three Tesla A16s. However, [8] does not focus on classification metrics such as accuracy since it is fully based on the same methods and dataset considered in [22]. Instead, the main issue of [8] was the time taken to classify the HS skin cancer images with the different GPUs, confirming how these devices are suitable to support medical diagnoses also in real-time.

In [31], a High Level Synthesis (HLS) approach was adopted to test an innovative ML model on different FPGAs, dissipating less energy than GPUs in processing HS images, and an average energy consumption (on eight HS images) of 8.8 J with the most performant platform (Virtex Ultrascale+) was reported. Once again, the energy consumption of this manuscript is lower.

All the works mentioned above adopted powerful processors, GPUs and FPGAs to perform HS image classification in the medical field and did not focus on the employment of ultra-low power platforms. Conversely, this paper employed a tiny device like the Arduino Nano 33 BLE Sense module (costing only a few dozens of euros), achieving classification results comparable to that of the literature at the expense of a very low power consumption. This suggests potential applicability in portable HS pre-screening scenarios where real-time outcomes are not critical.

Finally, a work concerning HSI classification on microcontrollers is proposed in [48]. Specifically, the authors developed a CNN using the TensorFlow Lite library to classify three HS datasets related to remote sensing. For this purpose, they employed some Commercial-Off-The-Shelf (COTS) microcontrollers achieving an overall accuracy ranging from 82.57% to 95.81%. However, the platforms

Table 5

Comparison with the state of the art. All the metrics are expressed in percentages.

	[29]	[30]	[22]	This work
Accuracy	63	70	97	76
Precision	70	N.A.	84	85
Recall	63	N.A.	86	76
Specificity	95	74	97	91
F1 score	65	N.A.	85	79

employed in [48] did not refer to an Arduino board like the one used in this paper. In addition, the HS data addressed in this manuscript are related to skin lesions, therefore a direct comparison is not feasible.

A last consideration concerns the reasons for employing this dataset. First, it allows a fair comparison with existing literature works employing the same dataset on different platforms including GPUs, many-core CPUs, and FPGAs. This aspect is important to evaluate the proposed TinyML implementation against existing methods and hardware employed in HS skin lesions inference. Second, to the best of our knowledge, no previous work addressed HS skin cancer classification entirely performed on tiny platforms. Therefore, the selected dataset represents a valid benchmark to demonstrate the feasibility of HS skin cancer portable pre-screening under strict computational constraints. Furthermore, the considered dataset features multiple lesion categories and pixel-level annotations. This aspect makes this dataset suitable for evaluating HS image classification methods in realistic clinical scenarios. Finally, publicly available labelled HS skin cancer datasets are really limited due to the complexity and cost of HS sensors, the expensive annotation process, and privacy issues. Although limited in size, the dataset considered in this work contains real images acquired from patients with different skin conditions and lesion types. In addition, the considered dataset was labelled by medical experts such as dermatologists and pathologists, ensuring reliable ground truths labels for supervised learning.

6. Conclusions

This work proposed a method to port a CNN model into the tiny Arduino Nano 33 BLE Sense board enabling a portable HS skin cancer pre-screening solution. The primary objective of this work was to investigate the feasibility of HS inference on an ultra-low-power microcontroller. Being the HS images acquired offline, the study focused exclusively on the computational stage, where HS data are processed and classified, without directly dealing with the acquisition process. However, the serial communication adopted to transfer the images from the PC to the board emulates a sequential acquisition scenario in which HS data could be directly acquired from a sensor and progressively fed to the microcontroller. In this context, the adopted pixel-wise approach is particularly suitable, as it enables processing data sequentially without requiring the storage of the whole HS image into external buffers or memories. This approach is compatible with streaming-based acquisition systems and allows to handle images of arbitrary size without incurring memory issues, although the inference time increases linearly with the number of pixels.

The CNN model has been built and trained using TensorFlow on the free access Google Colab Jupyter Notebook service. Then, the model has been converted into a smaller size version to make it compatible with the limited computational resources of the Arduino device through the TensorFlow Lite toolkit. Unlike typical TinyML demonstrations that focus on small sensor signals, the proposed system processes real HS images. This required repeated neural network inference over a large number of spectral vectors under strict RAM and Flash constraints. This aspect represents one of the main contributions of the work since many studies dealing with HS images typically adopt high performance computing platforms such as GPUs, FPGAs and many-core processors and do not incur memory requirements. Furthermore, the implementation required the design of a suitable processing pipeline capable of handling HS images under the limited RAM and Flash memory available on the Arduino Nano 33 BLE Sense. In particular, HS data are processed sequentially, only a limited portion of the data is stored at a time and inference is performed iteratively for each pixel. Although the tiny device employed in this work cannot compete with more powerful platforms such as GPUs in terms of classification time, results demonstrate HS skin cancer pre-screening feasibility on an ultra-low power microcontroller under strict computational resources. Nevertheless, since the sensing stage is not included in this study, the proposed approach

represents a proof-of-concept for HS skin lesion pre-screening on resource-constrained hardware, rather than a fully deployable clinical solution. The model developed in this research is simpler than other CNN architectures like AlexNet or VGGNet featuring more layers. The choice of limiting the number of convolutional and fully connected layers is linked to the resource-constrained Arduino device and to the intention of not incur memory issues. Future works will examine architectures with more layers and will determine how far tiny devices can go. Also, the integration of a real HS camera with the device will be examined in order to develop a system able to process HS data directly at the point of acquisition. This will allow a comprehensive analysis of the full system, including both sensing and computation, in terms of the overall cost and power consumption. Indeed, the acquisition system including the HS camera represents the most expensive and power-intensive part. This aspect represents an important direction for future research. Furthermore, since the considered dataset is limited in size, data augmentation techniques will be adopted to enlarge the training set variability, and larger datasets including more images related to BE and MM lesions will be considered, as they become available, to better generalize the segmentation of these lesions and to better validate the proposed method. Also, the following optimization strategies will be considered to reduce inference time: reduction of the input dimensionality (e.g., band selection or spectral compression), processing restricted to regions of interest (ROI), significantly reducing the number of pixels to be analyzed, and hardware acceleration using more advanced embedded platforms such as the STM32N6 series based on the Arm Cortex -M55. In addition, techniques such as knowledge distillation, transfer learning, and lightweight feature extraction will be evaluated to transfer the discriminative spectral representations learned by large foundation models into TinyML architectures. Finally, compact foundation models specifically designed to run efficiently on edge devices, together with emerging frameworks that facilitate the execution of pretrained foundation models on embedded microcontrollers will be considered.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work the author used ChatGPT (OpenAI) to assist with language refinement and improve the clarity of the manuscript. After using this tool, the author reviewed and edited the content as needed and takes full responsibility for the content of the published article.

CRediT authorship contribution statement

Bernardo Petracchi: Writing – review & editing, Writing – original draft, Validation, Supervision, Software, Project administration, Methodology, Formal analysis, Data curation, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

References

- [1] Lin, J., Zhu, L., Chen, W.-M., Wang, W.-C., & Han, S. (2024). *Tiny machine learning: progress and futures*. <https://doi.org/10.1109/MCAS.2023.3302182>.
- [2] M. Merenda, C. Porcaro, D. Iero, Edge machine learning for AI-enabled IoT devices: a review, *Sensors* 20 (9) (2020) 2533, <https://doi.org/10.3390/s20092533>.

- [3] R. Immonen, T. Hämäläinen, Tiny machine learning for resource-constrained microcontrollers, *J. Sens.* 2022 (2022) 1–11, <https://doi.org/10.1155/2022/7437023>.
- [4] Banbury, C.R., Reddi, V.J., Lam, M., Fu, W., Fazel, A., Holleman, J., Huang, X., Hurtado, R., Kanter, D., Lokhmotov, A., Patterson, D., Pau, D., Seo, J., Sieracki, J., Thakker, U., Verhelst, M., & Yadav, P. (2020). *Benchmarking TinyML systems: challenges and direction*.
- [5] TinyML for all developers with Edge Impulse. <https://www.hackster.io/news/tiny-ml-for-all-developers-with-edge-impulse-2cfbbcc14b90>, 2020.
- [6] C.V.F. Pereira, E.M. de Oliveira, A.D. de Souza, Machine learning applied to Edge computing and wearable devices for healthcare: systematic mapping of the literature, *Sensors* 24 (19) (2024) 6322, <https://doi.org/10.3390/s24196322>.
- [7] L. Barillaro, G. Agapito, M. Cannataro, Edge-based deep learning in medicine: classification of ECG signals, in: 2022 IEEE International Conference on Bioinformatics and Biomedicine (BIBM), 2022, pp. 2169–2174, <https://doi.org/10.1109/BIBM55620.2022.9995598>.
- [8] B. Petracchi, E. Torti, E. Marenzi, F. Leporati, Acceleration of hyperspectral skin cancer image classification through parallel machine-learning methods, *Sensors* 24 (5) (2024) 1399, <https://doi.org/10.3390/s24051399>.
- [9] M. Kamruzzaman, D.-W. Sun, *Introduction to hyperspectral imaging technology. Computer Vision Technology for Food Quality Evaluation*, Elsevier, Amsterdam, The Netherlands, 2016, pp. 111–139.
- [10] J.M. Meyer, R.F. Kokaly, E. Holley, Hyperspectral remote sensing of white mica: a review of imaging and point-based spectrometer studies for mineral resources, with spectrometer design considerations, *Remote Sens. Environ.* 275 (2022) 113000, <https://doi.org/10.1016/j.rse.2022.113000>.
- [11] T.H. Johansen, K. Møllersen, S. Ortega, H. Fabelo, A. Garcia, G.M. Callico, F. Godtlielsen, Recent advances in hyperspectral imaging for melanoma detection, *WIREs Comput. Stat.* 12 (1) (2020), <https://doi.org/10.1002/wics.1465>.
- [12] M. Tran, L. Ma, J.V. Litter, A.Y. Chen, B. Fei, Thyroid carcinoma detection on whole histologic slides using hyperspectral imaging and deep learning, in: R. M. Levenson, J.E. Tomaszewski, A.D. Ward (Eds.), *Medical Imaging 2022: Digital and Computational Pathology*, SPIE, 2022, p. 19, <https://doi.org/10.1117/12.2612963>.
- [13] H. Mahmood, M. Shaban, N. Rajpoot, S.A. Khurram, Artificial intelligence-based methods in head and neck cancer diagnosis: an overview, *Br. J. Cancer* 124 (12) (2021) 1934–1940, <https://doi.org/10.1038/s41416-021-01386-x>.
- [14] S. Trajanovski, C. Shan, P.J.C. Weijtmans, S.G.B. de Koning, T.J.M. Ruers, Tongue tumor detection in hyperspectral images using deep learning semantic segmentation, *IEEE Trans. Biomed. Eng.* 68 (4) (2021) 1330–1340, <https://doi.org/10.1109/TBME.2020.3026683>.
- [15] M.F. Avram, D.C. Lazăr, M.I. Mariș, S. Olariu, Artificial intelligence in improving the outcome of surgical treatment in colorectal cancer, *Front. Oncol.* 13 (2023), <https://doi.org/10.3389/fonc.2023.1116761>.
- [16] A. Zimmermann, H. Köhler, C. Chalopin, B. Jansen-Winkel, R. Nowotny, T. Schönherr, M. Mehdorn, K.L. Uttinger, R. Thieme, I. Gockel, Y. Moulla, The role of intraoperative hyperspectral imaging (HSI) in colon interposition after esophagectomy, *BMC Surg.* 23 (1) (2023) 47, <https://doi.org/10.1186/s12893-023-01946-3>.
- [17] G. Florimbi, H. Fabelo, E. Torti, S. Ortega, M. Marrero-Martin, G.M. Callico, G. Danese, F. Leporati, Towards real-time computing of intraoperative hyperspectral imaging for brain cancer detection using multi-GPU platforms, *IEEE Access* 8 (2020) 8485–8501, <https://doi.org/10.1109/ACCESS.2020.2963939>.
- [18] L. Rey-Barroso, S. Peña-Gutiérrez, C. Yáñez, F.J. Burgos-Fernández, M. Vilaseca, S. Royo, Optical technologies for the improvement of skin cancer diagnosis: a review, *Sensors* 21 (1) (2021) 252, <https://doi.org/10.3390/s21010252>.
- [19] S. Jiang, H. Li, Z. Jin, A visually interpretable deep learning framework for histopathological image-based skin cancer diagnosis, *IEEE J. Biomed. Health Inform.* 25 (5) (2021) 1483–1494, <https://doi.org/10.1109/JBHI.2021.3052044>.
- [20] M. Dildar, S. Akram, M. Irfan, H.U. Khan, M. Ramzan, A.R. Mahmood, S.A. Alsaiani, A.H.M. Saeed, M.O. Alraddadi, M.H. Mahnashi, Skin cancer detection: a review using deep learning techniques, *Int. J. Environ. Res. Public Health* 18 (10) (2021) 5479, <https://doi.org/10.3390/ijerph18105479>.
- [21] M. la Salvia, E. Torti, G. Lago, R. Leon, R. Gandolfi, G. Silveri, M. Rossella, G. Danese, P. Lago, F. Leporati, Hyperspectral imaging acquisition set-up for medical applications, in: N.J. Barnett, A.A. Gowen, H. Liang (Eds.), *Hyperspectral Imaging and Applications II*, SPIE, 2023, p. 13, <https://doi.org/10.1117/12.2647570>.
- [22] B. Petracchi, M. Gazzoni, E. Torti, E. Marenzi, F. Leporati, Machine learning-based classification of skin cancer hyperspectral images, *Procedia Comput. Sci.* 225 (2023) 2856–2865, <https://doi.org/10.1016/j.procs.2023.10.278>.
- [23] R. Leon, B. Martínez-Vega, H. Fabelo, S. Ortega, V. Melian, I. Castaño, G. Carretero, P. Almeida, A. Garcia, E. Quevedo, J.A. Hernandez, B. Clavo, G.M. Callico, Non-invasive skin cancer diagnosis using hyperspectral imaging for In-situ clinical support, *J. Clin. Med.* 9 (6) (2020) 1662, <https://doi.org/10.3390/jcm9061662>.
- [24] C. Tian, Y. Xu, Y. Zhang, Z. Zhang, H. An, Y. Liu, Y. Chen, H. Zhao, Z. Zhang, Q. Zhao, W. Li, Combining hyperspectral imaging techniques with deep learning to aid in early pathological diagnosis of melanoma, *Photodiagn. Photodyn. Ther.* 43 (2023) 103708, <https://doi.org/10.1016/j.pdpdt.2023.103708>.
- [25] Kazianka, H., Leitner, R., & Pilz, J. (2008). *Segmentation and classification of hyperspectral skin data* (pp. 245–252). https://doi.org/10.1007/978-3-540-78246-9_29.
- [26] V. Vinokurov, Y. Khristoforova, O. Myakinin, I. Bratchenko, A. Moryatov, A. Machikhin, V. Zakharov, Neural network classifier for hyperspectral images of skin pathologies, *J. Phys.: Conf. Ser.* 2127 (1) (2021) 012026, <https://doi.org/10.1088/1742-6596/2127/1/012026>.
- [27] A. Pardo, J.A. Gutiérrez-Gutiérrez, I. Lihacova, J.M. López-Higuera, O.M. Conde, On the spectral signature of melanoma: a non-parametric classification framework for cancer detection in hyperspectral imaging of melanocytic lesions, *Biomed. Opt. Express* 9 (12) (2018) 6283, <https://doi.org/10.1364/BOE.9.006283>.
- [28] H. Fabelo, V. Melian, B. Martínez, P. Beltrán, S. Ortega, M. Marrero, G.M. Callico, R. Sarmiento, I. Castano, G. Carretero, P. Almeida, A. Garcia, J.A. Hernandez, F. Godtlielsen, Dermatologic hyperspectral imaging system for skin cancer diagnosis assistance, in: 2019 XXXIV Conference on Design of Circuits and Integrated Systems (DCIS), 2019, pp. 1–6, <https://doi.org/10.1109/DCIS201949030.2019.8959869>.
- [29] M. la Salvia, E. Torti, R. Leon, H. Fabelo, S. Ortega, F. Balea-Fernandez, B. Martínez-Vega, I. Castaño, P. Almeida, G. Carretero, J.A. Hernandez, G. M. Callico, F. Leporati, Neural networks-based on-site dermatologic diagnosis through hyperspectral epidermal images, *Sensors* 22 (19) (2022) 7139, <https://doi.org/10.3390/s22197139>.
- [30] M. la Salvia, E. Torti, M. Gazzoni, E. Marenzi, R. Leon, S. Ortega, H. Fabelo, G. M. Callico, F. Leporati, Attention-based skin cancer classification through hyperspectral imaging, in: 2022 25th Euromicro Conference on Digital System Design (DSD), 2022, pp. 871–876, <https://doi.org/10.1109/DSD57027.2022.00122>.
- [31] E. Marenzi, E. Torti, G. Danese, F. Leporati, FPGA High level synthesis for the classification of skin tumors with hyperspectral images, in: 2022 11th Mediterranean Conference on Embedded Computing (MECO), 2022, pp. 1–4, <https://doi.org/10.1109/MECO55406.2022.9797211>.
- [32] N.E. Nevala, T. Baden, A low-cost hyperspectral scanner for natural imaging and the study of animal colour vision above and under water, *Sci. Rep.* 9 (2019) 10799, <https://doi.org/10.1038/s41598-019-47220-6>.
- [33] X. Li, Z. Chen, J. Wang, J. Jin, LeafSpec-Dicot: an accurate and portable hyperspectral imaging device for dicot leaves, *Sensors* 23 (2023) 3687, <https://doi.org/10.3390/s23073687>.
- [34] H. Mangotra, S. Srivastava, G. Jaiswal, R. Rani, A. Sharma, Hyperspectral imaging for early diagnosis of diseases: a review, *Expert Syst.* 40 (8) (2023) e13311, <https://doi.org/10.1111/exsy.13311>.
- [35] D. Hong, C. Li, N. Yokoya, et al., Hyperspectral imaging, *Nat. Rev. Methods Primers* 6 (2026) 19, <https://doi.org/10.1038/s43586-026-00470-x>.
- [36] X. Li, C. Li, G. Vivone, D. Hong, *Inf. Fusion* 125 (2026) 103334.
- [37] D. Hong, et al., SpectralGPT: spectral remote sensing foundation model, *IEEE Trans. Pattern Anal. Mach. Intell.* 46 (2024) 5227–5244.
- [38] C. Li, et al., Learning disentangled priors for hyperspectral anomaly detection: a coupling model-driven and data-driven paradigm, *IEEE Trans. Neural Netw. Learn. Syst.* 36 (2025) 6883–6896.
- [39] Bommasani, R., Hudson, D.A., Adeli, E., Altman, R., Arora, S., von Arx, S., Bernstein, M.S., Bohg, J., Bosselut, A., Brunskill, E., et al. On the opportunities and risks of foundation models. arXiv preprint arXiv:2108.07258, 2021.
- [40] X. Zhao, L. Wang, Y. Zhang, et al., A review of convolutional neural networks in computer vision, *Artif. Intell. Rev.* 57 (2024) 99, <https://doi.org/10.1007/s10462-024-10721-6>.
- [41] S. Jia, L. Zhijie, M. Xu, Q. Huang, J. Zhou, X. Jia, Q. Li, A lightweight convolutional neural network for hyperspectral image classification, *IEEE Trans. Geosci. Remote Sens.* 59 (2020) 4150–4163, <https://doi.org/10.1109/TGRS.2020.3014313>.
- [42] S. Yu, S. Jia, C. Xu, Convolutional neural networks for hyperspectral image classification, *Neurocomputing* 219 (2017) 88–98, <https://doi.org/10.1016/j.neucom.2016.09.010>. ISSN 0925-2312.
- [43] W. Hu, Y. Huang, L. Wei, F. Zhang, H. Li, Deep convolutional neural networks for hyperspectral image classification, *J. Sens.* 2015 (2015) 12, <https://doi.org/10.1155/2015/258619>, pages, 2015.
- [44] Q. Gao, S. Lim, X. Jia, Hyperspectral image classification using convolutional neural networks and multiple feature learning, *Remote Sens.* 10 (2018) 299, <https://doi.org/10.3390/rs10020299>.
- [45] Chan, R.; Kan, K.; Nikolova, M.; Plemmons, R. A two-stage method for spectral-spatial classification of hyperspectral images. <https://arxiv.org/abs/1806.00836>.
- [46] Model quantization using TensorFlow lite. <https://medium.com/sclable/model-quantization-using-tensorflow-lite-2fe6a171a90d>, 2020.
- [47] Understanding TF Lite and model optimization. <https://www.kaggle.com/code/asushma/understanding-tf-lite-and-model-optimization#Posttraining-Quantization>, 2022.
- [48] J.V. Hüttner, F. Viel, C.A. Zeferino, E.A. Bezerra, TinyML applied in hyperspectral image classification on COTS microcontroller, in: 2024 XIV Brazilian Symposium on Computing Systems Engineering (SBESC), 2024, pp. 1–6.